

Build, image, containerise

Day 2 - Afternoon

Running scenario: Thara Hospitals patient portal pilot

Niranga Dharmaratna
School of Computing & Engineering
National Institute of Business Management



CONCEPT 1



Tagging and the ledger

10 min

Three tags on everything, and the ledger first

- Every resource today: `Module=CC, Day=2, Owner=<student id>`
- A tag changes nothing about the resource; it lets you find it and cost it
- The ledger row is written **before** teardown, not after

Once a resource is deleted, the console can no longer tell you its id or its start time.

CONCEPT 2

What you are about to do that is wrong

A leaked access key is how the leaked-link incident could have been much worse.

10 min

Step 10 is the insecure method, deliberately

- You put a long-lived access key on the instance to reach S3
- Day 5 replaces it with an instance role. The contrast is the point
- Fenced in: off today, on for minutes on Day 4, deleted on Day 5

COMMON MISTAKE

Deactivating a key deletes it.

It is only switched off. Day 5 removes it.



A key that never expires, in a text file on a server.

Build, image, containerise



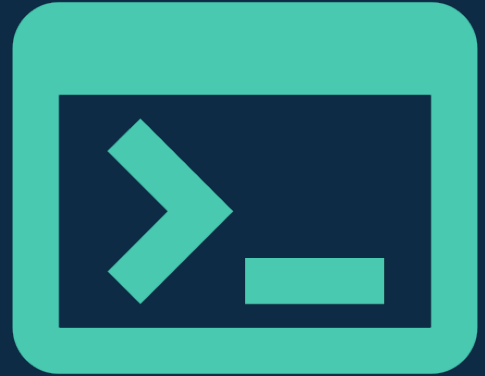
You build:

2

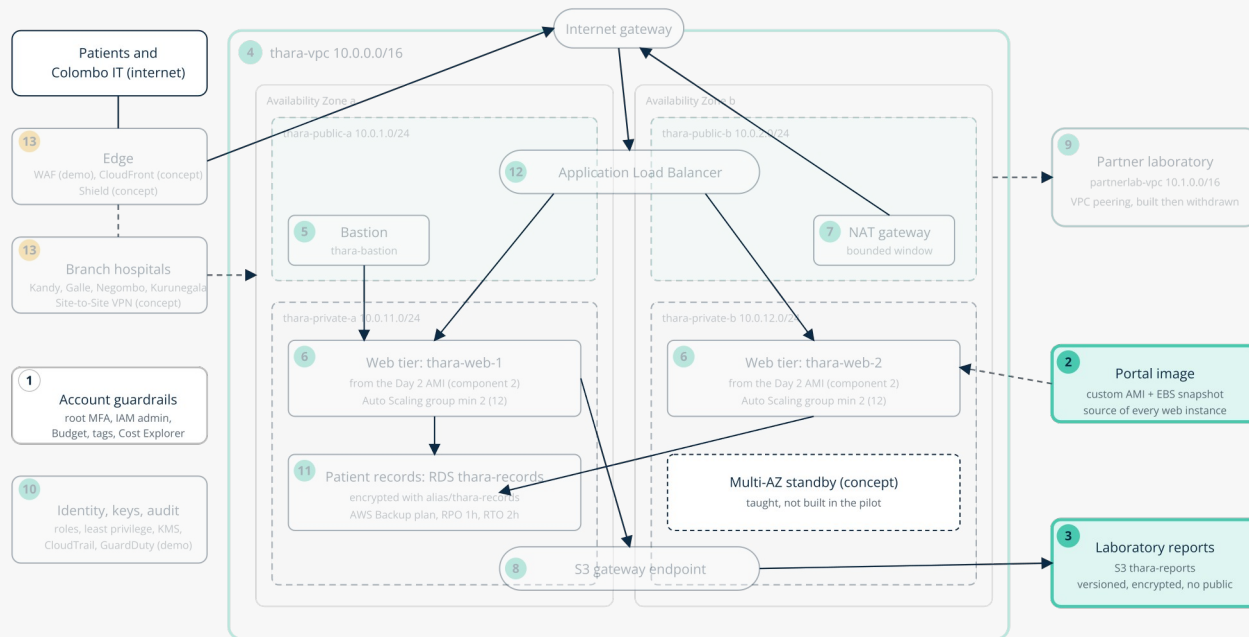
3

Estimated credit use: about USD 0.60

Niranga Dharmaratna
School of Computing & Engineering
National Institute of Business Management



The Thara architecture



TODAY

- 2** Patient portal web server and image
- 3** Laboratory reports bucket

KEY

- Built today
- Built earlier
- Not yet built

SEGMENT 1 - 30 MIN



Launch and connect

Cost: USD 0.011 per hour plus USD 0.005 per hour for IPv4

Check, then launch the web server



1

Confirm the region is `ap-south-1` and that you are the admin user, not root.

Expected: Mumbai, and `admin-<student id>`, in the top bar.

2

From Explore AWS, launch `thara-web-1`: Amazon Linux, t3.micro, new key pair `thara-key`, default VPC, SSH from My IP, HTTP from the internet, the three tags.

Expected: Running, with 2/2 checks passed after about two minutes.

`thara-key.pem` downloads once. Keep it safe: Day 3 needs it.

The user data for step 2

```
# paste under Advanced details, User data
#!/bin/bash
dnf install -y httpd
echo "<h1>Thara Hospitals portal (pilot)</h1>" \
  > /var/www/html/index.html
systemctl enable --now httpd
```

The script runs once, at first boot, as root. Nobody logs in to run it.

Connect, and load the page



3

Add an inbound SSH rule whose source is the prefix list ending `ec2-instance-connect`. Then Connect, EC2 Instance Connect. Browse to `http://` and the public IPv4 address.

Expected: A terminal as `ec2-user`, and the page Thara Hospitals portal (pilot).

The browser terminal comes from the Instance Connect service, not from your IP.

SEGMENT 2 - 35 MIN



EBS attach, format, snapshot

Cost: 8 GB gp3, under USD 0.01 for the afternoon

Attach a data volume and snapshot it

4

Create an 8 GiB gp3 volume in the instance's zone, tag it, attach it as `/dev/sdf`. Format, mount at `/data`, write `reports-index.txt`.

Expected: df shows 8.0G mounted on /data.

5

Snapshot the volume with the three tags. Write the snapshot id in your ledger.

Expected: The snapshot shows Completed.

```
$ lsblk
$ sudo mkfs -t xfs /dev/nvme1n1
$ sudo mkdir /data && sudo mount /dev/nvme1n1 /data
$ echo "Thara reports index" | sudo tee /data/reports-index.txt
```

SEGMENT 3 - 15 MIN



Create the AMI

Cost: snapshot storage, a few cents a month

Create the image

6

Actions, Image and templates, Create image, named `thara-portal-v1`. Remove the 8 GiB data volume from Instance volumes. Tag image and snapshots together.

Expected: The AMI moves from Pending to Available. Your terminal drops: the instance reboots.

```
# reconnect, then mount the data volume again
$ sudo mount /dev/nvme1n1 /data
$ cat /data/reports-index.txt
```

Leave the data volume in, and every server launched from this image gets an extra disk.

SEGMENT 4 - 35 MIN



Docker contrast

Cost: nil

Run a container beside the native server

7

Install Docker and run nginx mapped to port 8080. Add an inbound rule for TCP 8080 from My IP. Browse to port 80 and to port 8080.

Expected: Port 80: the Thara pilot page. Port 8080: Welcome to nginx!

```
$ sudo dnf install -y docker
$ sudo systemctl enable --now docker
$ sudo docker run -d --name web8080 -p 8080:80 nginx
```

Compare the container with the host

8

Compare processes, memory and the file-system view from the host and from inside the container. Record three differences.

Expected: nginx is a host process; no second OS uses memory; the container cannot see /data.

```
$ ps -ef | grep -E "httpd|nginx"
$ sudo docker top web8080
$ sudo docker stats --no-stream web8080
$ sudo docker exec web8080 cat /etc/os-release
$ sudo docker exec web8080 ls /data
```

SEGMENT 5 - 25 MIN



Reports bucket and the insecure upload

Cost: nil

Create the reports bucket



9

Create `thara-reports-<student id>` in `ap-south-1`. Leave ACLs disabled and Block all public access ticked. Add the three tags.

Expected: Permissions shows Block all public access: On.

Insecure, deliberately: the long-lived key

10

IAM, your admin user, Security credentials, Create access key, Command Line Interface. Download the .csv. Configure the CLI on the instance and upload the file.

Expected: An upload line naming `thara-reports-<student id>`, then the object listed.

```
$ aws configure
# <bucket> is thara-reports-<student id>
$ aws s3 cp /data/reports-index.txt s3://<bucket>/
$ aws s3 ls s3://<bucket>
```

The secret now sits in plain text on the server. Ledger note now; never in a screenshot.

SEGMENT 6 - 20 MIN



Ledger, teardown, evidence

Cost: nil

Prove it, then break it



PROVE IT

- `curl http://localhost` returns the pilot page; `curl http://localhost:8080` returns nginx
- `aws s3 ls` on the bucket, from the instance, lists the object
- The AMI `thara-portal-v1` shows Available

BREAK IT

- Remove the HTTP rule from the security group and reload the page
- Observe: a timeout, while `curl http://localhost` still works
- Restore the rule. Note which layer the failure happened at

Tear down and record



TEARDOWN

- Remove the 8080 rule. Stop `thara-web-1`; do not terminate it
- Detach and delete the 8 GiB volume; keep the snapshot and the AMI
- Deactivate the access key; do not delete it. Keep the bucket

COST

Estimated credit use: about USD 0.60

FOR YOUR EVIDENCE PACK

- Screenshot of the pilot page and the nginx page side by side
- The three recorded VM-versus-container differences
- AMI id and snapshot id; bucket name with Block Public Access shown on
- Ledger row with the note about the access key

Common mistakes



COMMON MISTAKE

The container is inside a separate machine.

It is a process on the same kernel.



COMMON MISTAKE

Deactivating a key deletes it.

It does not. Day 5 removes it.

Before the next session



READ

Amazon VPC User Guide

How Amazon VPC works. Then Wittig and Wittig, the VPC chapter.

SELF-CHECK

CLF-C02 bank

Domain 3 storage set, 10 items.

PQ2 is due.

TASK

Carve 10.0.0.0/16

Four subnets across two AZs, two public and two private. Say why each is which.

On paper. Bring it to Day 3.

Questions & Discussion

Next: the network. You build `thara-vpc` and prove the image inside it.



Disclaimer & Copyright



The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability of the content within this presentation.

Cloud services, prices and free-tier terms change. Figures in this presentation were correct when prepared; check the provider's current documentation before relying on them.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites, and I am not responsible for their content.

Thara Hospitals (Pvt) Ltd is a fictional organisation created for teaching. AWS and Amazon Web Services are trademarks of Amazon.com, Inc. or its affiliates; this material is not affiliated with or endorsed by AWS.



© 2026 Niranga Dharmaratna, School of Computing & Engineering, National Institute of Business Management. Teaching content licensed under CC BY-NC-SA 4.0. Images and artworks used for explanation may contain third-party material used under fair dealing for education.

Revision 2a8e67f